Sztuczna inteligencja i inżynieria wiedzy - Lista 1

Emilia Kosior

25 marca 2026

1 Wstęp

Projekt obejmuje implementację i optymalizację algorytmów wyszukiwania połączeń oraz planowania tras wielopunktowych w oparciu o rzeczywiste dane rozkładowe Kolei Dolnośląskich (format GTFS 2026).

2 Zadanie 1: Wyszukiwanie ścieżek (Dijkstra i A*)

2.1 Algorytm Dijkstry

Algorytm Dijkstry stanowił fundament Zadania 1(a). Został zaimplementowany jako bazowa metoda wyznaczania najkrótszej ścieżki w czasie.

- **Cel:** Minimalizacja całkowitego czasu podróży (różnica między czasem przyjazdu a startu).
- **Działanie:** Systematyczne przeszukiwanie grafu z wykorzystaniem kolejki priorytetowej, wybierając krawędzie pozwalające na najwcześniejsze dotarcie do kolejnych węzłów.
- **Zastosowanie:** Służy jako punkt odniesienia (baseline) dla wydajności algorytmu A*.

2.2 Algorytmy A* – Funkcje Kosztu i Heurystyki

Algorytm A* wykorzystuje funkcję celu: $f(n) = g(n) + h(n)$. W projekcie zaimplementowano dwie wersje optymalizacji:

Kryterium	Koszt rzeczywisty $g(n)$	Funkcja heurystyczna $h(n)$
Czas (t)	Suma sekund od startu.	Czas dojazdu w linii prostej (100 km/h).
Przesiadki (p)	Liczba przesiadek + suma sekund od startu podzielona przez 10^{12}.	kara za brak bezpośredniego połączenia do celu + dystans Haversine podzielony przez 10^4.

Uwaga: W trybie przesiadek (p) w celach demonstracyjnych celowo nie ograniczono maksymalnego czasu oczekiwania na pociąg. Dzięki temu algorytm znajduje połączenia nawet kilka miesięcy do przodu, jeśli wymagają mniejszej liczby przesiadek. Zachowanie to można modyfikować, zmieniając wagę czasu w funkcji $g(n)$.

2.3 Optymalizacje Algorytmu A*

W celu uzyskania wysokiej wydajności zastosowano:

- **Sterta Binarna (`heapq`):** Zapewnia wyciąganie stanu o najniższym koszcie w czasie $O(\log n)$.
- **Agregacja Stacji:** Mapowanie przez `parent_station` eliminuje zbędne przesiadki między peronami.
- **System Cache (`@lru_cache`):** Klasa `Service` buforuje wyniki sprawdzania kalendarza, co drastycznie przyspiesza weryfikację dostępności pociągu.
- **Rozwiązanie Problemu Relaksacji:** Wprowadzono system dopuszczający wiele linii dla tego samego kosztu w węźle, co zapobiega odrzucaniu optymalnych tras bezpośrednich przez trasy z przesiadką.

3 Zadanie 2: Tabu Search

Tabu Search to metaheurystyka lokalnego przeszukiwania, której istotą jest wykorzystanie listy niedozwolonych ruchów (listy tabu) by zablokować powrót do niedawno odwiedzonych stanów, co zapobiega utknięciu w optimach lokalnych oraz eliminuje ryzyko wystąpienia cykli. Algorytm w każdej iteracji zawsze wybiera najlepszego dostępnego sąsiada – nawet jeśli powoduje on pogorszenie funkcji celu – o ile dany ruch nie jest aktualnie zabroniony i spełnia kryterium aspiracji. Dzięki takiemu podejściu metoda ta pozwala na efektywną eksplorację przestrzeni stanów, co w problemach takich jak optymalizacja tras kolejowych umożliwia skuteczne szeregowanie przystanków przy dynamicznie zmieniających się kosztach zależnych od czasu.

- **Mechanizm Tabu:** Lista przechowuje nazwy stacji, blokując cykliczne powroty do tych samych układów.
- **Rozmiar T:** Dynamiczny, zdefiniowany jako $0.7 \times \text{len}(L)$.

- **Aspiracja:** Dopuszczenie ruchu Tabu, jeśli skutkuje nowym rekordem globalnym.
- **Próbkowanie:** Losowanie podzbioru sąsiedztwa w celu przyspieszenia iteracji.

4 Przykłady wywołania funkcji

```
from Algorithms.Dijkstra import *
from Algorithms.Tabu_search import *

graph = FullGTFSGraph("../train_data")
start_station = "wrocław główny"
end_station = "zielona góra główna"
start_datetime = datetime.now()

L = ["Legnica", "Wałbrzych Główny", "Jaworzyna Śląska", "Lubin"]

print_results(start_datetime, dijkstra_time(graph, start_station,
    end_station, start_datetime))
print("------------------------------")

print_results(start_datetime, modal_a_star(graph, start_station,
    end_station, start_datetime, 't'))
print("------------------------------")

print_results(start_datetime, modal_a_star(graph, start_station,
    end_station, start_datetime, 'p'))
print("------------------------------")

print_results(start_datetime, tabu_search(graph, start_station, L,
    start_datetime, 't', iterations=20, sample_size=3))
```

Output:

```
Cost: 8893, Calculation time: 0:00:00.438461
Shortest path found after 0:00:00.438461, takes 2:28:13 with 0
    transfers.
Route:
  D20 from Wrocław Główny (20:54:00) to Zielona Góra Główna
    (23:21:00)
------------------------------
Cost: 8893, Calculation time: 0:00:00.226857
Shortest path found after 0:00:00.226857, takes 2:28:13 with 0
    transfers.
Route:
  D20 from Wrocław Główny (20:54:00) to Zielona Góra Główna
    (23:21:00)
------------------------------
Shortest path found after 0:00:00.018603, takes 87 days, 11:27:13
    with 0 transfers.
Route:
  Wait for 88 days
```

```
  KD Premium Nadmorski from Wrocław Główny (6:22:00) to Zielona Góra Główna (8:20:00)
-----------------------------
Cost: 0.0, Calculation time: 0:00:00.018603
Cost: 35173, Calculation time: 0:00:44.560810
Shortest path found after 0:00:44.560810, takes 9:46:13 with 3 transfers.
Route:
  D20 from Wrocław Główny (20:54:00) to Wrocław Muchobór (20:59:00)
  D11 from Wrocław Muchobór (21:35:00) to Legnica (23:01:00)
  D12 from Legnica (23:25:00) to Wrocław Główny (1 day, 0:11:00)
  D6 from Wrocław Główny (3:31:00) to Wrocław Główny (6:39:00)
```

5 Problemy Implementacyjne

1. **Skomplikowanie formatu GTFS:** Przetworzenie powiązań między plikami rozkładowymi na wydajny graf czasozależny wymagało długiej analizy wszystkich dostępnych parametrów w dokumentach. Wielokrotnie konieczny był powrót do dokumentacji plików i poprawki w skrypcie importującym graf.

2. **Relaksacja przy przesiadkach:** Klasyczny A* gubił trasy bezpośrednie na rzecz szybszych przesiadkowych. Rozwiązano to przez dopuszczanie do sterty ścieżek o tym samym koszcie (tolerancja < 1), pod warunkiem, że wjeżdżają na stację unikalną linią - dzięki temu przeanalizowany zostanie przypadek wjazdu na tą stację każdą możliwą linią o minimalnej liczbie przesiadek.

6 Technologie

Projekt wykorzystuje biblioteki: `datetime`, `heapq`, `math`, `csv` oraz `functools`. Obliczenia dystansu oparto na wzorze Haversine:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$